

SCD FINAL EXAM: MASTER REVISION GUIDE

1. SOFTWARE DESIGN PROCESS (CHAPTER 2)

Definition: Transforming customer requirements (SRS) into a form implementable by code.

The 3 Phases of Software Design:

1. **Interface Design:** Treating the system as a black box; focusing on how it interacts with users/outside systems.
2. **Architectural Design:** Defining major components and how they communicate.
3. **Detailed Design:** Specifying internal algorithms and exact logic of each component.

The 7 Software Design Concepts (Crucial Headings):

1. **Abstraction:** Hiding complex details to reduce complexity.
2. **Modularity:** Dividing the system into smaller, independent parts.
3. **Architecture:** The structural blueprint of the software.
4. **Refinement:** Elaborating a system step-by-step to remove errors.
5. **Pattern:** A repeated solution to a recurring problem.
6. **Information Hiding:** Securing data inside a module so others cannot access it.
7. **Refactoring:** Reconstructing code to reduce complexity without changing external behavior.

2. SOLID DESIGN PRINCIPLES

The 5 Principles:

1. **Single Responsibility Principle (SRP):** A class should only have one job (one reason to change).
2. **Open/Closed Principle (OCP):** Open for extension, closed for modification.
3. **Liskov Substitution Principle (LSP):** Subclasses must be substitutable for their base classes without breaking code.
4. **Interface Segregation Principle (ISP):** Clients should not be forced to depend on methods they do not use.
5. **Dependency Inversion Principle (DIP):** High-level modules should depend on abstractions, not concrete details.

- **[PAST PAPER INTEGRATION] SOLID & Green IT:**

Applying SOLID saves energy. SRP allows scaling only the specific microservices under heavy load (saving server RAM). ISP reduces network payload sizes (saving bandwidth energy). DIP allows swapping power-hungry hardware for green cloud alternatives easily.

3. DESIGN PATTERNS

Creational Patterns (Object Creation):

1. **Singleton:** Ensures a class has only exactly one instance (Good for DB connections; Bad because it creates a global state bottleneck).
 2. **Factory:** Provides an interface for creating objects, letting subclasses decide the type (Good for loose coupling; Bad because it adds class overhead).
- **[PAST PAPER INTEGRATION] Singleton vs Factory:** Use Factory for dynamic object creation, but use Singleton inside the Factory to limit expensive resource creation (hybrid approach).

Behavioral Patterns (Object Communication):

1. **Observer:** "Subscriber" model. One subject changes state, all listening observers auto-update (e.g., Weather app).
2. **Strategy:** Encapsulates interchangeable algorithms into separate classes, allowing them to be swapped at runtime (e.g., swapping Credit Card for PayPal).

4. CODE QUALITY & METRICS

Code Metrics (Quantitative Measurements):

1. **Lines of Code (LOC):** Measures project size.
2. **Cyclomatic Complexity:** Measures independent paths (if/else). 1-10 is good, >20 requires refactoring.
3. **Code Coverage:** Percentage of code tested.
4. **Coupling:** Degree of dependency between modules (Keep it LOW).
5. **Cohesion:** How closely related tasks are inside one module (Keep it HIGH).
6. **Technical Debt:** The future cost of choosing a quick, messy solution today.

Code Smells (Symptoms of bad code):

Long Method, Large Class, Duplicated Code, Primitive Obsession, Feature Envy, Data Clumps, Inconsistent Naming, Lazy Class, Speculative Generality, Switch Statements.

Anti-Patterns (Bad Solutions):

God Object, Spaghetti Code, Shotgun Surgery, Golden Hammer, Magic Numbers, Copy-Paste Programming.

- **[PAST PAPER INTEGRATION] Metrics & Green IT:** Optimizing code (lowering complexity, high cohesion) reduces CPU execution time. This stops servers from overheating, saves electricity, and delays the need to buy new servers (reducing electronic waste).

5. REFACTORING & READABILITY

Readability Best Practices:

- Meaningful Naming Conventions, Keep Functions Small, Use Proper Indentation, Avoid Magic Numbers, Keep it DRY (Don't Repeat Yourself).
- **Commenting:** Comment *Why*, not *What*. Write self-explanatory code first.

Refactoring Techniques (The 14 Methods):

1. Extract Method
2. Inline Method
3. Extract Class
4. Inline Temp
5. Replace Temp with Query
6. Replace Conditional with Polymorphism (Fixes Switch Statements)
7. Move Method
8. Rename Method/Variable
9. Remove Dead Code
10. Replace Magic Numbers with Constants
11. Introduce Parameter Object (Fixes Data Clumps)
12. Decompose Conditional
13. Encapsulate Field
14. Replace Type Code

6. FUNDAMENTALS OF TESTING (CHAPTER 5)

Key Objectives:

1. **Verification:** Are we building the product right? (Code works perfectly).
2. **Validation:** Are we building the right product? (Customer actually wants it).
3. **Defect Detection & Risk Mitigation.**

Levels of Testing (Sequential Order):

1. **Unit Testing:** Testing individual methods/classes.
2. **Integration Testing:** Testing if modules work together.
3. **System Testing:** Testing the entire system as a whole.
4. **Acceptance Testing:** User tests the software to accept delivery (UAT).

Test-Driven Development (TDD):

- **Cycle:** Red (Write failing test) -> Green (Write minimum code to pass) -> Refactor (Clean up code).

7. DEBUGGING (CHAPTER 5)

Phases of Debugging:

1. Identify the Bug
2. Analyze the Problem
3. Fix the Bug
4. Test the Fix
5. Document the Fix

Debugging Techniques:

Print Debugging (Logging), Interactive Debugging (Breakpoints), Rubber Duck Debugging, Binary Search (Divide & Conquer), Backtracking.

8. EXCEPTION HANDLING & LOGGING (CHAPTER 6)

Handling Mechanism:

`try` (risky code), `catch` (handle error), `finally` (always runs for cleanup), `throw` (manual error trigger).

Exception Best Practices:

Use Specific Exceptions, Avoid Silent Failures, Throw Early Catch Late (Fail Fast), Use Custom Exceptions.

Logging Levels & Best Practices:

- **Levels:** DEBUG, INFO, WARN, ERROR, FATAL.
- **Practices:** Use Structured Logging (JSON), Use Correlation IDs (to track requests across servers), Configure Log Rotation.
- **[PAST PAPER INTEGRATION] Logging & Green IT:** Using Log Rotation and logging only ERRORS (instead of every INFO) saves massive hard drive storage and network bandwidth, extending hardware life and reducing e-waste.

9. SOFTWARE DEPLOYMENT (CHAPTER 8)

Deployment Strategies:

- **Monolithic vs. Microservices:** Deploying as one giant block vs. tiny independent services.
- **Blue-Green Deployment:** Two identical servers. Deploy to Green, test, then instantly switch all users from Blue to Green (Zero downtime).
- **Canary Deployment:** Releasing new version to a small 5% subset of users first.
- **Rolling Deployment:** Replacing servers phase-by-phase.

CI/CD Pipeline Sequence:

Source Control -> Automated Testing -> Automated Builds -> Automated Deployments -> Monitoring and Alerts.

- **[PAST PAPER INTEGRATION] Docker & Kubernetes:**

Docker is Containerization (puts code in a standard environment so it runs exactly the same anywhere). Kubernetes is Orchestration (manages thousands of containers, provides auto-scaling, load balancing, and self-healing).

10. SOFTWARE MAINTENANCE & EVOLUTION

The 4 Types of Maintenance (Crucial):

1. **Corrective:** Fixing post-deployment bugs (e.g., App crash).
2. **Adaptive:** Modifying for a new environment (e.g., iOS update).
3. **Perfective:** Adding new features to meet user needs (e.g., adding Apple Pay).
4. **Preventive:** Changing internal code to prevent future errors (e.g., refactoring).

11. CODE SALVAGING & CONFIGURATION MANAGEMENT

Code Salvaging (Reusing Code):

- Complete Module Salvage (reusing a whole module).
- Partial Salvage (reusing specific functions).

Configuration Management (CM) Activities:

1. **Version Control:** Tracking code changes over time (Git).
2. **Change Control:** Reviewing and approving requested changes.
3. **Build Management:** Compiling code into executables.
4. **Release Management:** Managing software versions delivered to the user.